

Task 3 (Easy): Maximum Profit Analyzer

Problem Description

Given daily profit/loss values, find the maximum possible profit obtainable from a contiguous sequence of days using Kadane's Algorithm.

Input Format

- First line contains integer N.
- Second line contains N integers.

Output Format

Print maximum subarray sum.

Constraints

- $1 \leq N \leq 10^5$

Sample Input

```
8
-2 -3 4 -1 -2 1 5 -3
```

Sample Output

```
7
```

Program:

```
package Taskss;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
import java.util.StringTokenizer;

public class task3 {

    public static void main(String[] args) throws IOException {

        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));

        String line1 = reader.readLine();

        if (line1 == null || line1.trim().isEmpty()) return;

        int n = Integer.parseInt(line1.trim());

        long[] arr = new long[n];

        // Parse array input efficiently using StringTokenizer
```

```

String line2 = reader.readLine();

if (line2 != null) {

    StringTokenizer st = new StringTokenizer(line2);

    for (int i = 0; i < n && st.hasMoreTokens(); i++) {

        arr[i] = Long.parseLong(st.nextToken());

    }

}

// Kadane's Algorithm

long maxSoFar = arr[0];

long currentMax = arr[0];

for (int i = 1; i < n; i++) {

    // Decide whether to add current element to existing subarray or start fresh from current element

    currentMax = Math.max(arr[i], currentMax + arr[i]);

    // Track the global maximum profit found so far

    maxSoFar = Math.max(maxSoFar, currentMax);

}

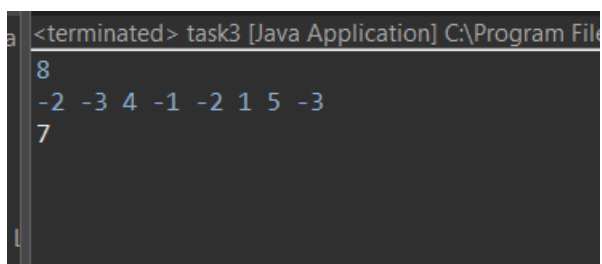
System.out.println(maxSoFar);

}

}

```

Output:



```

a <terminated> task3 [Java Application] C:\Program File
8
-2 -3 4 -1 -2 1 5 -3
7

```